GUIA VISUAL 059

FUNÇÕES RETORNO E ESCOPO

*O inventário contava pacotes com `return "$n"`. Na instalação enxuta imprimia **180** — certo. Na estação com o `kali-linux-everything`, 4.412 pacotes, imprimiu **60**. Sem erro, sem aviso: o valor não coube no canal. Uma função tem **dois** caminhos para devolver informação, e o script escolheu o estreito.*

059
GUIAS LINUX

OS DOIS CANAIS DE SAÍDA — DADO ≠ VEREDITO

FUNÇÃO
`contar_pacotes`
Um **comando novo**, indistinguível de
`ls`
para quem chama — e com precedência sobre o programa externo.

SAÍDA PADRÃO — `n=$(contar_pacotes)`
Texto de qualquer tamanho. Cabe `4412`, cabe uma tabela inteira, cabe um caminho com espaços. Custo: o `$( )` cria **subshell** — variável atribuída lá dentro morre junto — e remove as quebras de linha do fim.

CÓDIGO DE SAÍDA — `contar_pacotes; $?`
Inteiro de 0 a 255, e só. O `return` responde *deu certo?*, não *qual é o resultado?*. `return` sem argumento devolve o código do **último comando** — o que se quer numa função-predicado como `usuario_existe() { id "$1" >/dev/null 2>&1; }`.

return 0	return 1	return 255	return 256	return 300	return 4412
0	1	255	0 falha vira sucesso	44	60 o bug da abertura

COMO DEVOLVER DADOS DE VERDADE

`n=$(contar_pacotes)`
Idiomática. A função imprime, o chamador captura. Comporta-se como função de verdade — ao custo de um processo por chamada, o que pesa em laço apertado.

`RESULTADO=… ; contar_pacotes`
Global por convenção. Sem subshell, a mais rápida. Cria acoplamento invisível: quem lê a chamada não vê que a variável mudou. Nome em MAIÚSCULAS e comentário na linha acima.

`local -n destino="$1"`
Referência nomeada (Bash 4.3+): o chamador escolhe onde o resultado vai parar. Prefixe o nome interno (`__destino`) — passar variável homônima dispara erro de referência circular.

`local x=$(cmd)` **engole o erro do comando** (SC2155): `local` é um comando, e o `$?` que sobra é o **dele** — quase sempre zero. Declare numa linha, atribua na seguinte: `local v; v=$(cmd) || return 1`. Vale igual para `declare`, `export` e `readonly`.

ESCOPO **DINÂMICO**: VALE A PILHA, NÃO O TEXTO

nível superior
`segredo` **não existe** aqui — nem antes nem depois da chamada.

↓ chama

`externa() { local segredo="abc"; … }`
Dona da variável. Ao retornar, o valor anterior é restaurado.

↓ chama

`interna() { echo "$segredo"; }`
Vê `abc` sem receber nada como argumento.

Em C ou Python o escopo é **léxico** — definido por onde a função está escrita. No shell é definido por **quem chamou quem** em execução. Sem `local`, toda variável da função é global; e um `local i` sobrescreve o `i` de qualquer função chamada abaixo. Em biblioteca, nada de `i`, `n`, `tmp` ou `arquivo`.

ENCERRAR, ENVOLVER, E QUANDO PROMOVER A FUNÇÃO A SCRIPT

as três armadilhas do dia a dia

```
# exit encerra o SCRIPT; return encerra so a FUNCAO. Biblioteca nunca chama exit.
morrer() { printf 'erro: %s\n' "$*" >&2; exit 1; }   # unica excecao legitima

# 'command' pula a funcao e chama o programa real -- sem ele, recursao infinita
rm() { command rm -i "$@"; }     # legitimo no ~/.bashrc, armadilha dentro de script

# $0 continua sendo o SCRIPT dentro da funcao; o nome da funcao esta em FUNCNAME
$ quem() { echo "$0 | ${FUNCNAME[0]} | chamado por ${FUNCNAME[1]:-<script>}"; }
./ctx.sh | quem | chamado por externa

# sob set -e, a funcao usada como CONDICAO tem o set -e suspenso dentro dela
if valida; then …   # erro interno nao aborta: exija return explicito em cada saida
```

SITUAÇÃO	ESCOLHA	SITUAÇÃO	ESCOLHA
Alterar diretório ou variáveis da sessão	**Função** — script não consegue	Chamada por `cron` ou `systemd`	**Script** próprio